



Bismillah General Store

Complete Step-by-Step Documentation

A comprehensive guide for beginner developers to understand every file in this React + Express + MongoDB e-commerce grocery store project.

React.js

Express.js

MongoDB

Node.js

Version 1.0 | 2026

Table of Contents

- 1 Project Overview
- 2 Technology Stack
- 3 Project Structure
- 4 Frontend: Entry Files
- 5 Frontend: Configuration
- 6 Frontend: Context Providers
- 7 Frontend: Customer Pages
- 8 Frontend: Auth Pages
- 9 Frontend: Dashboard Pages
- 10 Frontend: Components
- 11 Frontend: Utilities
- 12 Backend: Server & Config

13 Backend: Middleware

14 Backend: Models

15 Backend: Routes

16 How to Run the Project

17 Common Concepts for Beginners

18 Tips for New Developers

1. Project Overview

The **Bismillah General Store** is a full-stack e-commerce application designed for an online grocery store. It allows customers to browse products, add items to their cart, place orders, and track their purchase history. Admin users can manage products, categories, orders, users, and view analytics through a comprehensive dashboard.

Key Features

- **Customer Features:** Browse products, search/filter, shopping cart, checkout with multiple payment options, order tracking, chat support
- **Admin Features:** Dashboard with analytics, product management with CRUD operations, category management, order processing, user management, stock management, sales reports, bill records, recycle bin for deleted items
- **Authentication:** JWT-based authentication with role-based access control (admin vs customer)
- **File Upload:** Cloudinary integration for product image storage
- **Real-time Chat:** Admin-customer communication system

For Beginners

This project demonstrates a real-world e-commerce application with both frontend (React) and backend (Express + MongoDB) components. It is an excellent learning resource for understanding full-stack web development.

How It Works (High Level)

1. The **React frontend** sends HTTP requests to the backend API using Axios

2. The **Express server** receives requests, validates JWT tokens, and processes business logic
3. The server communicates with **MongoDB** through Mongoose to store and retrieve data
4. Images are uploaded to **Cloudinary** and URLs are stored in MongoDB
5. The frontend receives JSON responses and updates the UI accordingly

2. Technology Stack

Frontend Technologies



React.js

UI library for building user interfaces with component-based architecture



React Router

Client-side routing for navigation between pages



Recharts

Charting library for dashboard analytics visualization



SCSS/SASS

CSS preprocessor for styling with variables and nesting



React Hot Toast

Toast notifications for user feedback



Axios

HTTP client for API requests with interceptors

Backend Technologies



Node.js

JavaScript runtime for server-side execution



Express.js

Web framework for building REST APIs



MongoDB

NoSQL database for flexible data storage



Mongoose

ODM library for MongoDB object modeling



JWT

JSON Web Tokens for stateless authentication



Cloudinary

Cloud service for image upload and management

Technology Summary Table

Technology	Purpose	Location
React.js	Frontend UI rendering	client/src/
Express.js	API server and routing	server/
MongoDB + Mongoose	Database operations	server/models/
JWT	User authentication	server/middleware/auth.js
Cloudinary	Image storage	server/config/cloudinary.js
Axios	HTTP requests	client/src/config/api.js
React Router	Client-side routing	client/src/pages/Routes.jsx
Recharts	Data visualization	client/src/pages/Dashboard/Home/

3. Project Structure

The project follows a standard MERN stack structure with separate client and server directories:

```
Bismillah-General-Store/
├── client/                                # React frontend application
│   ├── public/                           # Static assets
│   │   └── index.html                    # HTML template
│   ├── src/                              # Source code
│   │   ├── main.jsx                     # Application entry point
│   │   ├── App.jsx                      # Root component
│   │   ├── App.scss                     # Global styles
│   │   ├── config/                      # Configuration files
│   │   │   ├── api.js                   # Axios API client
│   │   │   ├── global.jsx               # Global utilities
│   │   │   ├── firebase.jsx             # Firebase config (placeholder)
│   │   │   └── supabase.jsx             # Supabase config (placeholder)
│   │   ├── context/                     # React Context providers
│   │   │   ├── AppProvider.jsx          # Combined context provider
│   │   │   ├── Auth.jsx                 # Authentication context
│   │   │   └── Cart.jsx                 # Shopping cart context
│   │   ├── pages/                       # Page components
│   │   │   ├── Routes.jsx               # Main routing configuration
│   │   │   ├── Frontend/                # Customer-facing pages
│   │   │   │   ├── index.jsx            # Frontend layout wrapper
│   │   │   │   ├── Home/index.jsx       # Home page
│   │   │   │   ├── Shop/index.jsx       # Shop page
│   │   │   │   ├── ProductDetails/index.jsx
│   │   │   │   ├── Cart/index.jsx       # Shopping cart
│   │   │   │   ├── Checkout/index.jsx
│   │   │   │   ├── Orders/index.jsx     # Customer orders
│   │   │   │   ├── About/index.jsx
│   │   │   └── Contact/index.jsx
│   │   ├── Auth/                        # Authentication pages
│   │   │   ├── index.jsx                # Auth routes wrapper
│   │   └── Login/index.jsx
```

```

| | | | | └─ Register/index.jsx
| | | | | └─ ForgotPassword/index.jsx
| | | | └─ Dashboard/ # Admin dashboard pages
| | | |   └─ index.jsx # Dashboard layout with sidebar
| | | |   └─ Home/index.jsx # Analytics with charts
| | | |   └─ Products/index.jsx
| | | |   └─ Products/RecycleBin.jsx
| | | |   └─ Orders/index.jsx
| | | |   └─ Categories/index.jsx
| | | |   └─ Sales/index.jsx
| | | |   └─ Bills/index.jsx
| | | |   └─ Stock/index.jsx
| | | |   └─ Users/index.jsx
| | | |   └─ Chat/index.jsx
| | | |   └─ RecycleBin/index.jsx
| | | └─ components/ # Reusable components
| | |   └─ Header/
| | |     └─ index.jsx
| | |     └─ Navbar.jsx
| | |   └─ Footer/
| | |     └─ index.jsx
| | |     └─ Copyright.jsx
| | |   └─ Chat/ChatWidget.jsx
| | |   └─ Misc/
| | |     └─ ScreenLoader.jsx
| | |     └─ NoPage.jsx
| | |     └─ ProtectedRoute.jsx
| | └─ utils/ # Utility functions
| |   └─ format.js
| |   └─ chat.js
| |   └─ recycleBin.js
| └─ package.json
|
└─ server/ # Express backend application
  └─ index.js # Server entry point
  └─ config/
    └─ db.js # MongoDB connection
    └─ cloudinary.js # Cloudinary config
  └─ middleware/
    └─ auth.js # JWT authentication
    └─ upload.js # File upload handling
  └─ models/

```

```
| | └─ User.js
| | └─ Product.js
| | └─ Category.js
| | └─ Order.js
| | └─ Chat.js
| | └─ StockMovement.js
| └─ routes/
| | └─ auth.js
| | └─ products.js
| | └─ categories.js
| | └─ orders.js
| | └─ users.js
| | └─ dashboard.js
| | └─ stock.js
| | └─ chat.js
| | └─ upload.js
└─ package.json
└─ README.md
```

Understanding the Structure

The project uses a **monorepo structure** where both client and server are in the same repository. The `client/` folder contains the React application, while the `server/` folder contains the Express API.

4. Frontend Guide - Entry Files

client/src/main.jsx

What This File Does

This is the entry point of the React application. It creates the root React component and renders it into the DOM. It wraps the app with BrowserRouter for routing and AppProvider for state management.

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';
import { AppProvider } from './context/AppProvider';
import './App.scss';

// Get the root DOM element from index.html
const root = ReactDOM.createRoot(document.getElementById('root'));

// Render the application with providers
root.render(
  <React.StrictMode>
    <BrowserRouter>
      <AppProvider>
        <App />
      </AppProvider>
    </BrowserRouter>
  </React.StrictMode>
);
```

Line-by-Line Explanation

- **Lines 1-5:** Import necessary libraries and components

- **Line 8:** Create a root element using React 18's new createRoot API
- **Lines 11-19:** Render the App component wrapped in three providers:
 - `React.StrictMode` - Helps identify potential problems in development
 - `BrowserRouter` - Enables client-side routing (URL-based navigation)
 - `AppProvider` - Provides authentication and cart state to all components

[-> App.jsx](#)[-> AppProvider.jsx](#)[-> App.scss](#)[client/src/App.jsx](#)

What This File Does

The main application component that renders the Routes component and includes global elements like the chat widget and toast notifications.

```
import { Toaster } from 'react-hot-toast';
import Routes from './pages/Routes';
import ChatWidget from './components/Chat/ChatWidget';
import ScreenLoader from './components/Misc/ScreenLoader';
import { useApp } from './context/AppProvider';

function App() {
  const { loading } = useApp();

  // Show loading screen while initial data is fetched
  if (loading) {
    return <ScreenLoader />;
  }

  return (
    <>
      <Routes />
      <ChatWidget />
      <Toaster position="top-right" toastOptions={{
        duration: 3000,
        style: { background: '#333', color: '#fff' } },
      }} />
    </>
  );
}
```

```
);
}

export default App;
```

Line-by-Line Explanation

- **Lines 1-5:** Import toast notifications, routes, chat widget, and loading screen
- **Line 7:** Define the App functional component
- **Line 8:** Get loading state from AppProvider context using the custom useApp hook
- **Lines 11-13:** Show loading screen while initial data (categories, user verification) is being fetched
- **Lines 15-24:** Render the main application with Fragment wrapper:
 - **Routes** - Contains all page routes
 - **ChatWidget** - Floating chat button for customers to contact support
 - **Toaster** - Toast notification system for success/error messages

-> Routes.jsx

-> ChatWidget.jsx

-> ScreenLoader.jsx

<- AppProvider.jsx

client/src/App.scss

What This File Does

Contains global SCSS styles including CSS variables for theming, resets, typography, and common utility classes used throughout the application.

```
// CSS Variables for consistent theming
:root {
  --primary-color: #1d3557;
  --secondary-color: #2a9d8f;
  --accent-color: #e76f51;
  --light-bg: #f1faee;
```

```
--text-dark: #333;
--text-light: #666;
--border-color: #ddd;
--shadow: 0 2px 10px rgba(0,0,0,0.1);
--border-radius: 8px;
--transition: all 0.3s ease;
}

// Global Reset
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  color: var(--text-dark);
  line-height: 1.6;
  background-color: var(--light-bg);
}

// Common Button Styles
.btn {
  padding: 10px 20px;
  border: none;
  border-radius: var(--border-radius);
  cursor: pointer;
  font-weight: 600;
  transition: var(--transition);

  &--primary {
    background: var(--primary-color);
    color: white;
    :hover { opacity: 0.9; }
  }
  &--secondary {
    background: var(--secondary-color);
    color: white;
  }
}

.container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 0 20px;
}
```

Key Concepts

CSS Variables: Defined in `:root` for consistent theming. Use `var(--variable-name)` to reference them. **SCSS Nesting:** Use `&` to reference parent selectors.

5. Frontend Guide - Configuration

client/src/config/api.js

What This File Does

Configures Axios for making HTTP requests to the backend API. Sets up base URL, request interceptors for adding auth tokens, and response interceptors for global error handling.

```
import axios from 'axios';
import toast from 'react-hot-toast';

// Create Axios instance with base configuration
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:5000/api',
  timeout: 10000,
  headers: { 'Content-Type': 'application/json' },
});

// REQUEST INTERCEPTOR - runs before every request
// Automatically adds the JWT token to the Authorization header
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('token');
    if (token) {
      config.headers.Authorization = Bearer ;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// RESPONSE INTERCEPTOR - runs after every response
// Handles errors globally (e.g., 401 unauthorized)
api.interceptors.response.use(
  (response) => response,
```

```

(error) => {
  if (error.response?.status === 401) {
    localStorage.removeItem('token');
    window.location.href = '/login';
  }
  const message = error.response?.data?.message || 'Something went wrong';
  toast.error(message);
  return Promise.reject(error);
}
);

export default api;

```

How Interceptors Work

Interceptor	When It Runs	What It Does
Request	Before every API call	Adds JWT token to headers
Response (Error)	After any failed request	Shows toast error, handles 401 redirect

<- All API calls use this

-> server/routes/*

client/src/config/global.jsx

What This File Does

Provides global constants, role definitions, order status enums, payment methods, and utility functions used throughout the application.

```

// User roles
export const ROLES = { ADMIN: 'admin', CUSTOMER: 'customer' };

// Order status options
export const ORDER_STATUS = {

```

```
PENDING: 'pending',
CONFIRMED: 'confirmed',
PROCESSING: 'processing',
SHIPPED: 'shipped',
DELIVERED: 'delivered',
CANCELLED: 'cancelled',
};

// Available payment methods
export const PAYMENT_METHODS = [
  'Cash on Delivery', 'Credit Card', 'Debit Card',
  'JazzCash', 'EasyPaisa',
];

// Pagination defaults
export const PAGINATION = { DEFAULT_PAGE: 1, DEFAULT_LIMIT: 10 };
```

client/src/config/firebase.jsx

What This File Does

Placeholder configuration for Firebase integration (currently not active).
Prepared for future Firebase authentication if needed.

Note

This is a placeholder file. To use Firebase, replace the placeholder values with your actual Firebase project credentials and uncomment the initialization code.

client/src/config/supabase.jsx

What This File Does

Placeholder configuration for Supabase integration (currently not active).
Prepared for future Supabase database and auth if needed.

6. Frontend Guide - Context Providers

React Context is a way to share state between components without passing props through every level. This project uses three contexts: AppProvider (combined), Auth, and Cart.

client/src/context/AppProvider.jsx

What This File Does

Combines multiple context providers into a single wrapper component. Provides authentication state, cart state, categories, and loading state to the entire application.

```
import { createContext, useContext, useState, useEffect } from 'react';
import { AuthProvider, useAuth } from './Auth';
import { CartProvider, useCart } from './Cart';
import api from '../config/api';

// Create the App Context
const AppContext = createContext();

// Custom hook to use App context from any component
export const useApp = () => useContext(AppContext);

// Inner component that has access to Auth and Cart contexts
function AppProviderInner({ children }) {
  const [loading, setLoading] = useState(true);
  const [categories, setCategories] = useState([]);
  const { user, token } = useAuth();

  useEffect(() => { fetchCategories(); }, []);

  const fetchCategories = async () => {
    try {
      const response = await api.get('/categories');
```

```
        setCategories(response.data);
    } catch (error) {
        console.error('Error fetching categories:', error);
    } finally {
        setLoading(false);
    }
};

const value = { loading, categories, user, token };

return (
    <AppContext.Provider value={value}>
        {children}
    </AppContext.Provider>
);
}

// Main provider that nests Auth and Cart providers
export function AppProvider({ children }) {
    return (
        <AuthProvider>
            <CartProvider>
                <AppProviderInner>{children}</AppProviderInner>
            </CartProvider>
        </AuthProvider>
    );
}
```

Context Composition Pattern

Notice how AppProvider nests AuthProvider and CartProvider. The inner AppProviderInner can access both Auth and Cart contexts because it is rendered inside them. This pattern makes it easy to wrap the entire app in main.jsx.

[-> Auth.jsx](#)[-> Cart.jsx](#)[-> main.jsx](#)[client/src/context/Auth.jsx](#)

What This File Does

Manages all authentication state including user login, logout, registration, and JWT token verification. Provides auth state and methods to all components.

```
import { createContext, useContext, useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import toast from 'react-hot-toast';
import api from '../config/api';

const AuthContext = createContext();
export const useAuth = () => useContext(AuthContext);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(localStorage.getItem('token'));
  const [loading, setLoading] = useState(true);
  const navigate = useNavigate();

  // On mount: if token exists, verify it with the server
  useEffect(() => {
    if (token) { verifyToken(); }
    else { setLoading(false); }
  }, []);

  const verifyToken = async () => {
    try {
      const response = await api.get('/auth/verify');
      setUser(response.data.user);
    } catch (error) {
      logout(); // Token invalid - clear it
    } finally {
      setLoading(false);
    }
  };

  const login = async (email, password) => {
    const response = await api.post('/auth/login', { email, password });
    const { token, user } = response.data;
    localStorage.setItem('token', token);
    setToken(token);
    setUser(user);
    toast.success('Login successful!');
    navigate(user.role === 'admin' ? '/dashboard' : '/');
  };

  const register = async (userData) => {
    await api.post('/auth/register', userData);
    toast.success('Registration successful! Please login.');
```

```
const logout = () => {
  localStorage.removeItem('token');
  setToken(null);
  setUser(null);
  navigate('/login');
};

return (
  <AuthContext.Provider value={{ user, token, loading, login, register,
logout }}>
    {children}
  </AuthContext.Provider>
);
}
```

Authentication Flow

Function	Purpose	API Endpoint	Storage
<code>login()</code>	Authenticates user	POST /auth/login	JWT in localStorage
<code>register()</code>	Creates new account	POST /auth/register	None (redirects to login)
<code>logout()</code>	Clears session	None (client-side)	Removes token
<code>verifyToken()</code>	Validates token on mount	GET /auth/verify	Reads token from localStorage

-> api.js (HTTP calls)

-> server/routes/auth.js

<- AppProvider.jsx

client/src/context/Cart.jsx

What This File Does

Manages shopping cart state including adding items, removing items, updating quantities, and calculating totals. Persists cart in localStorage so it survives page refreshes.

```
import { createContext, useContext, useState, useEffect } from 'react';
import toast from 'react-hot-toast';

const CartContext = createContext();
export const useCart = () => useContext(CartContext);

export function CartProvider({ children }) {
  // Initialize cart from localStorage (persists across refreshes)
  const [cart, setCart] = useState(() => {
    const saved = localStorage.getItem('cart');
    return saved ? JSON.parse(saved) : [];
  });

  // Save cart to localStorage whenever it changes
  useEffect(() => {
    localStorage.setItem('cart', JSON.stringify(cart));
  }, [cart]);

  const addToCart = (product, quantity = 1) => {
    setCart(prev => {
      const existing = prev.find(item => item._id === product._id);
      if (existing) {
        // If already in cart, increase quantity
        return prev.map(item =>
          item._id === product._id
            ? { ...item, quantity: item.quantity + quantity }
            : item
        );
      }
      // Otherwise add new item
      return [...prev, { ...product, quantity }];
    });
    toast.success('Added to cart!');
  };

  const removeFromCart = (productId) => {
    setCart(prev => prev.filter(item => item._id !== productId));
  };

  const updateQuantity = (productId, quantity) => {
    if (quantity < 1) { removeFromCart(productId); return; }
    setCart(prev => prev.map(item =>
```

```

        item._id === productId ? { ...item, quantity } : item
    ));
};

const clearCart = () => setCart([]);

// Computed values
const total = cart.reduce((sum, item) => sum + item.price * item.quantity, 0);
const itemCount = cart.reduce((sum, item) => sum + item.quantity, 0);

return (
    <CartContext.Provider value={{ cart, addToCart, removeFromCart,
updateQuantity, clearCart, total, itemCount }}>
        {children}
    </CartContext.Provider>
);
}

```

Cart Functions Explained

Function	What It Does	Toast?
<code>addToCart(product, qty)</code>	Adds product or increases quantity if already exists	Yes
<code>removeFromCart(id)</code>	Removes a product completely from cart	No
<code>updateQuantity(id, qty)</code>	Sets exact quantity (removes if < 1)	No
<code>clearCart()</code>	Empties the entire cart	No
<code>total</code>	Computed total price of all items	-
<code>itemCount</code>	Total number of items in cart	-

-> localStorage (persistence)

<- AppProvider.jsx

<- Cart Page

<- ProductDetails Page

7. Frontend Guide - Customer Pages

client/src/pages/Routes.jsx

What This File Does

Defines all application routes and maps URL paths to their corresponding page components. Handles public routes, protected routes (require login), and admin routes (require admin role).

```
import { Routes, Route } from 'react-router-dom';
import { useAuth } from '../context/Auth';
import ProtectedRoute from '../components/Misc/ProtectedRoute';

// Import all page components...
import FrontendLayout from './Frontend';
import Home from './Frontend/Home';
import Shop from './Frontend/Shop';
// ... more imports ...

function AppRoutes() {
  const { user } = useAuth();

  return (
    <Routes>
      {/* PUBLIC FRONTEND ROUTES */}
      <Route element={<FrontendLayout />}>
        <Route path="/" element={<Home />} />
        <Route path="/shop" element={<Shop />} />
        <Route path="/product/:id" element={<ProductDetails />} />

        {/* PROTECTED ROUTES - require login */}
        <Route element={<ProtectedRoute />}>
          <Route path="/cart" element={<Cart />} />
          <Route path="/checkout" element={<Checkout />} />
          <Route path="/orders" element={<Orders />} />
        </Route>
      </Routes>
    )
  }
}
```

```

    </Route>

    { /* AUTH ROUTES */ }
    <Route element={<AuthLayout />}>
      <Route path="/login" element={<Login />} />
      <Route path="/register" element={<Register />} />
    </Route>

    { /* ADMIN DASHBOARD ROUTES - require admin role */ }
    <Route element={<ProtectedRoute isAdmin />}>
      <Route path="/dashboard" element={<DashboardLayout />}>
        <Route index element={<DashboardHome />} />
        <Route path="products" element={<Products />} />
        <Route path="orders" element={<OrdersManagement />} />
        { /* ... more dashboard routes ... */ }
      </Route>
    </Route>

    { /* 404 ROUTE */ }
    <Route path="*" element={<NoPage />} />
  </Routes>
);
}

```

Route Summary Table

Path	Component	Access	Description
/	Home	Public	Landing page with featured products
/shop	Shop	Public	Browse all products with filters
/product/:id	ProductDetails	Public	Single product view
/cart	Cart	Protected	Shopping cart
/checkout	Checkout	Protected	Payment and shipping

Path	Component	Access	Description
/orders	Orders	Protected	Order history
/login	Login	Public	User login
/register	Register	Public	New account creation
/dashboard	DashboardHome	Admin	Analytics overview
/dashboard/products	Products	Admin	Product CRUD
/dashboard/orders	Orders	Admin	Order management
/dashboard/categories	Categories	Admin	Category CRUD
/dashboard/users	Users	Admin	User management

client/src/pages/Frontend/index.jsx

What This File Does

Layout wrapper for all customer-facing pages. Includes Header at the top, main content area (where child routes render), and Footer at the bottom.

```
import { Outlet } from 'react-router-dom';
import Header from '../../../components/Header';
import Footer from '../../../components/Footer';

function FrontendLayout() {
  return (
    <div className="frontend-layout">
      <Header />
```

```

        <main className="main-content">
          <Outlet /> { /* Child route renders here */}
        </main>
        <Footer />
      </div>
    );
  }
}

```

About Outlet

`<Outlet />` is a React Router component that renders the child route's element. When you navigate to `/shop`, the `Shop` component renders inside the `Outlet`. This is how nested layouts work.

client/src/pages/Frontend/Home/index.jsx

What This File Does

The landing page of the store. Displays a hero banner, featured product categories, and a selection of featured products. Fetches data from the API on mount.

```

import { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import api from '../../../config/api';

function Home() {
  const [featuredProducts, setFeaturedProducts] = useState([]);
  const [categories, setCategories] = useState([]);

  useEffect(() => { fetchData(); }, []);

  const fetchData = async () => {
    const [productsRes, categoriesRes] = await Promise.all([
      api.get('/products?featured=true&limit=8'),
      api.get('/categories')
    ]);
  };
}

```

```

    setFeaturedProducts(productsRes.data.products);
    setCategories(categoriesRes.data);
  };

  return (
    <div className="home-page">
      <section className="hero-section">
        <h1>Welcome to Bismillah General Store</h1>
        <p>Your one-stop shop for quality groceries</p>
        <Link to="/shop" className="btn btn-primary">Shop Now</Link>
      </section>

      <section className="categories-section">
        <h2>Shop by Category</h2>
        <div className="categories-grid">
          {categories.map(category => (
            <Link key={category._id} to={"/shop?category="}>
              <div className="category-card">
                <img src={category.image} alt={category.name} />
                <h3>{category.name}</h3>
              </div>
            </Link>
          ))}
        </div>
      </section>

      <section className="featured-section">
        <h2>Featured Products</h2>
        <div className="products-grid">
          {featuredProducts.map(product => (
            /* Render product cards */
          ))}
        </div>
      </section>
    </div>
  );
}

```

The Home page makes two parallel API calls using `Promise.all()` to fetch featured products and categories simultaneously for better performance.

client/src/pages/Frontend/Shop/index.jsx

What This File Does

Displays all products with filtering by category, text search, and sorting. Uses URL search params so filters persist in the URL.

```
import { useState, useEffect } from 'react';
import { useSearchParams } from 'react-router-dom';
import api from '../../../config/api';
import { useApp } from '../../../context/AppProvider';

function Shop() {
  const [searchParams, setSearchParams] = useSearchParams();
  const [products, setProducts] = useState([]);
  const [filters, setFilters] = useState({
    search: searchParams.get('search') || '',
    category: searchParams.get('category') || '',
    sort: 'newest',
  });

  const { categories } = useApp();

  useEffect(() => { fetchProducts(); }, [filters]);

  const fetchProducts = async () => {
    const params = new URLSearchParams();
    if (filters.search) params.append('search', filters.search);
    if (filters.category) params.append('category', filters.category);
    params.append('sort', filters.sort);
    const response = await api.get(`/products?`);
    setProducts(response.data.products);
  };

  return (
    <div className="shop-page">
      <input type="text" placeholder="Search products..."
        value={filters.search}
        onChange={(e) => setFilters({ ...filters, search: e.target.value
      </div>
      <aside className="filters">
        {categories.map(cat => (
          <button key={cat._id}
            className={filters.category === cat._id ? 'active' : ''}
            onClick={() => setFilters({ ...filters, category: cat._id
        </button>
      </aside>
      <div className="products-grid">
        {products.map(product => { /* Product cards */})}
      </div>
    </div>
  )
}
```

```
);  
}
```

client/src/pages/Frontend/ProductDetails/index.jsx

What This File Does

Displays detailed information about a single product including image, name, price, description, stock status, and an add-to-cart button with quantity selector.

```
import { useState, useEffect } from 'react';  
import { useParams } from 'react-router-dom';  
import api from '../../../config/api';  
import { useCart } from '../../../context/Cart';  
  
function ProductDetails() {  
  const { id } = useParams(); // Gets :id from URL  
  const [product, setProduct] = useState(null);  
  const [quantity, setQuantity] = useState(1);  
  const { addToCart } = useCart();  
  
  useEffect(() => { fetchProduct(); }, [id]);  
  
  const fetchProduct = async () => {  
    const response = await api.get(/products/);  
    setProduct(response.data);  
  };  
  
  if (!product) return <div>Loading...</div>;  
  
  return (  
    <div className="product-details">  
      <img src={product.image} alt={product.name} />  
      <h1>{product.name}</h1>  
      <p className="price">Rs. {product.price}</p>  
      <p>{product.description}</p>  
      <div className="quantity-selector">  
        <button onClick={() => setQuantity(Math.max(1, quantity - 1))}>-</button>  
        <span>{quantity}</span>  
        <button onClick={() => setQuantity(quantity + 1)}>+</button>  
      </div>  
    </div>  
  );  
}
```

```

        <button onClick={() => addToCart(product, quantity)}>Add to
        Cart</button>
      </div>
    );
  }

```

-> api.js (fetch product)

-> Cart.jsx (addToCart)

<- URL param :id

client/src/pages/Frontend/Cart/index.jsx

What This File Does

Displays the shopping cart with all items, their quantities, prices, and a running total. Provides quantity controls and a checkout button.

```

import { Link } from 'react-router-dom';
import { useCart } from '../../../context/Cart';
import { formatPrice } from '../../../utils/format';

function Cart() {
  const { cart, removeFromCart, updateQuantity, total, itemCount } = useCart();

  if (cart.length === 0) {
    return (
      <div className="empty-cart">
        <h2>Your cart is empty</h2>
        <Link to="/shop">Continue Shopping</Link>
      </div>
    );
  }

  return (
    <div className="cart-page">
      <h1>Shopping Cart ({itemCount} items)</h1>
      {cart.map(item => (
        <div key={item._id} className="cart-item">
          <img src={item.image} alt={item.name} />
          <h3>{item.name}</h3>
          <p>{formatPrice(item.price)}</p>
          <button onClick={() => updateQuantity(item._id, item.quantity -
1)}>-</button>

```

```

        <span>{item.quantity}</span>
        <button onClick={() => updateQuantity(item._id, item.quantity +
1)}>+</button>
        <button onClick={() =>
removeFromCart(item._id)}>Remove</button>
      </div>
    )))
    <div className="cart-summary">
      <h2>Total: {formatPrice(total)}</h2>
      <Link to="/checkout">Proceed to Checkout</Link>
    </div>
  </div>
);
}

```

Other Frontend Pages (Summary)

client/src/pages/Frontend/Checkout/index.jsx

Checkout Page

Handles the checkout process with shipping address form, payment method selection (Cash on Delivery, JazzCash, etc.), order summary, and place order button. Creates an order via POST /api/orders.

client/src/pages/Frontend/Orders/index.jsx

Orders Page

Displays the logged-in customer's order history. Fetches orders via GET /api/orders/mine and shows order ID, date, items, total, and status for each order.

```
client/src/pages/Frontend/About/index.jsx
```

About Page

Static page displaying information about Bismillah General Store - mission, values, and store description.

```
client/src/pages/Frontend/Contact/index.jsx
```

Contact Page

Contact form for customer inquiries. Includes name, email, subject, and message fields. Submits to POST /api/contact or shows store contact information.

8. Frontend Guide - Auth Pages

client/src/pages/Auth/index.jsx

What This File Does

Layout wrapper for authentication pages (login, register, forgot password).
Provides a centered card layout with the store branding.

```
import { Outlet } from 'react-router-dom';

function AuthLayout() {
  return (
    <div className="auth-layout">
      <div className="auth-container">
        <div className="auth-header">
          <h1>Bismillah General Store</h1>
        </div>
        <Outlet /> {/* Login, Register, or ForgotPassword renders here */}
      </div>
    </div>
  );
}
```

client/src/pages/Auth/Login/index.jsx

What This File Does

Login form with email/password authentication. Uses the useAuth hook's login function. Redirects admin users to /dashboard and customers to / on success.

```
import { useState } from 'react';
import { Link } from 'react-router-dom';
import { useAuth } from '../../../context/Auth';

function Login() {
  const [formData, setFormData] = useState({ email: '', password: '' });
  const [loading, setLoading] = useState(false);
  const { login } = useAuth();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    try {
      await login(formData.email, formData.password);
    } catch (error) {
      // Error is handled by the API interceptor (toast shown)
    } finally {
      setLoading(false);
    }
  };

  return (
    <form className="auth-form" onSubmit={handleSubmit}>
      <h2>Login</h2>
      <input type="email" placeholder="Email"
        value={formData.email}
        onChange={(e) => setFormData({ ...formData, email: e.target.value
        required />
      <input type="password" placeholder="Password"
        value={formData.password}
        onChange={(e) => setFormData({ ...formData, password:
e.target.value }}>
        required />
      <button type="submit" disabled={loading}>
        {loading ? 'Logging in...' : 'Login'}
      </button>
      <p><Link to="/forgot-password">Forgot Password?</Link></p>
      <p>Don't have an account? <Link to="/register">Register</Link></p>
    </form>
  );
}
```

The login flow: User submits form -> `login()` from Auth context -> POST /api/auth/login -> JWT token stored in localStorage -> Redirect based on role.

client/src/pages/Auth/Register/index.jsx

What This File Does

Registration form for new customers. Collects name, email, password, and phone number. Calls the register function from Auth context.

```
import { useState } from 'react';
import { Link } from 'react-router-dom';
import { useAuth } from '../../../context/Auth';

function Register() {
  const [formData, setFormData] = useState({
    name: '', email: '', password: '', phone: '',
  });
  const { register } = useAuth();

  const handleSubmit = async (e) => {
    e.preventDefault();
    await register(formData);
  };

  return (
    <form className="auth-form" onSubmit={handleSubmit}>
      <h2>Create Account</h2>
      <input type="text" placeholder="Full Name"
        value={formData.name}
        onChange={(e) => setFormData({ ...formData, name: e.target.value
      ))}

      required />
      <input type="email" placeholder="Email"
        value={formData.email}
        onChange={(e) => setFormData({ ...formData, email: e.target.value
      ))}

      required />
      <input type="tel" placeholder="Phone Number"
        value={formData.phone}
        onChange={(e) => setFormData({ ...formData, phone: e.target.value
      ))}

      required />
    </form>
  );
}
```

```
        <input type="password" placeholder="Password"
            value={formData.password}
            onChange={(e) => setFormData({ ...formData, password:
e.target.value })}
            required />
        <button type="submit">Create Account</button>
        <p>Already have an account? <Link to="/login">Login</Link></p>
    </form>
  );
}
```

client/src/pages/Auth/ForgotPassword/index.jsx

What This File Does

Allows users to request a password reset by entering their email address. Sends a reset link to the provided email.

9. Frontend Guide -

Dashboard Pages

The dashboard is the admin-only section of the application. It has a sidebar navigation and multiple management pages.

client/src/pages/Dashboard/index.jsx

What This File Does

The dashboard layout component. Renders a sidebar with navigation links and a main content area where dashboard child routes render.

```
import { Outlet, NavLink } from 'react-router-dom';
import { useAuth } from '../../../context/Auth';

const sidebarLinks = [
  { path: '/dashboard', label: 'Dashboard', icon: '🏠' },
  { path: '/dashboard/products', label: 'Products', icon: '📦' },
  { path: '/dashboard/orders', label: 'Orders', icon: '📦' },
  { path: '/dashboard/categories', label: 'Categories', icon: '👉' },
  { path: '/dashboard/sales', label: 'Sales', icon: '💰' },
  { path: '/dashboard/bills', label: 'Bills', icon: '📄' },
  { path: '/dashboard/stock', label: 'Stock', icon: '📦' },
  { path: '/dashboard/users', label: 'Users', icon: '👤' },
  { path: '/dashboard/chat', label: 'Chat', icon: '💬' },
  { path: '/dashboard/recycle-bin', label: 'Recycle Bin', icon: '🗑️' },
];

function DashboardLayout() {
  const { user, logout } = useAuth();

  return (
    <div className="dashboard-layout">
      <aside className="sidebar">
        <div className="sidebar-header">
          <h2>Admin Panel</h2>
        </div>
```

```

    <nav className="sidebar-nav">
      {sidebarLinks.map(link => (
        <NavLink
          key={link.path}
          to={link.path}
          end={link.path === '/dashboard'}
          className={({ isActive }) => isActive ? 'active' : ''}
        >
          {link.icon} {link.label}
        </NavLink>
      ))}
    </nav>
    <div className="sidebar-footer">
      <p>Welcome, {user?.name}</p>
      <button onClick={logout}>Logout</button>
    </div>
  </aside>
  <main className="dashboard-content">
    <Outlet />
  </main>
</div>
);
}

```

The sidebar uses `NavLink` which automatically adds an "active" class to the current route's link.

client/src/pages/Dashboard/Home/index.jsx

What This File Does

Dashboard analytics page. Displays key metrics (total sales, orders, users, products) and charts showing sales trends and order status distribution using Recharts.

```

import { useState, useEffect } from 'react';
import { BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, PieChart, Pie, Cell }
from 'recharts';
import api from '../../../config/api';

function DashboardHome() {

```

```

const [stats, setStats] = useState({
  totalSales: 0,
  totalOrders: 0,
  totalUsers: 0,
  totalProducts: 0,
  recentOrders: [],
  salesData: [],
  orderStatusData: [],
});

useEffect(() => { fetchStats(); }, []);

const fetchStats = async () => {
  const response = await api.get('/dashboard/stats');
  setStats(response.data);
};

return (
  <div className="dashboard-home">
    <h1>Dashboard</h1>

    {/* Stats Cards */}
    <div className="stats-grid">
      <div className="stat-card">
        <h3>Total Sales</h3>
        <p>Rs. {stats.totalSales.toLocaleString()}</p>
      </div>
      <div className="stat-card">
        <h3>Total Orders</h3>
        <p>{stats.totalOrders}</p>
      </div>
      <div className="stat-card">
        <h3>Users</h3>
        <p>{stats.totalUsers}</p>
      </div>
      <div className="stat-card">
        <h3>Products</h3>
        <p>{stats.totalProducts}</p>
      </div>
    </div>

    {/* Sales Bar Chart */}
    <BarChart width={600} height={300} data={stats.salesData}>
      <CartesianGrid strokeDasharray="3 3" />
      <XAxis dataKey="month" />
      <YAxis />
      <Tooltip />
      <Bar dataKey="sales" fill="#2a9d8f" />
    </BarChart>

    {/* Order Status Pie Chart */}
    <PieChart width={400} height={300}>
      <Pie data={stats.orderStatusData} dataKey="value" nameKey="name" />
    </PieChart>
  </div>
)

```

```
        </PieChart>
      </div>
    );
  }
```

client/src/pages/Dashboard/Products/index.jsx

What This File Does

Full product management page with a table listing all products, search functionality, add/edit modal forms with image upload, and soft delete (move to recycle bin).

Key features: CRUD operations via API, image upload to Cloudinary, category selection dropdown, stock quantity management, featured product toggle.

client/src/pages/Dashboard/Products/RecycleBin.jsx

What This File Does

Shows products that have been soft-deleted. Admin can restore products back to active status or permanently delete them.

client/src/pages/Dashboard/Orders/index.jsx

What This File Does

Order management page for admins. Lists all orders with customer info, items, total, and status. Allows updating order status (pending, confirmed, processing, shipped, delivered, cancelled).

```
client/src/pages/Dashboard/Categories/index.jsx
```

What This File Does

Category management with CRUD operations. Admin can add new categories with name and image, edit existing ones, and delete categories.

```
client/src/pages/Dashboard/Sales/index.jsx
```

What This File Does

Displays sales records and reports. Shows total sales, sales by period, and allows filtering by date range.

```
client/src/pages/Dashboard/Bills/index.jsx
```

What This File Does

Bill records management. Tracks bills related to orders with payment status and details.

```
client/src/pages/Dashboard/Stock/index.jsx
```

What This File Does

Stock management page. Tracks product inventory levels, shows stock movements (in/out), and allows manual stock adjustments with reasons.

```
client/src/pages/Dashboard/Users/index.jsx
```

What This File Does

User management for admins. Lists all registered users, allows viewing user details, and managing user roles and status.

```
client/src/pages/Dashboard/Chat/index.jsx
```

What This File Does

Admin chat support interface. Allows admins to view and respond to customer chat messages in real-time.

```
client/src/pages/Dashboard/RecycleBin/index.jsx
```

What This File Does

Main recycle bin page showing all soft-deleted items (products, categories, etc.) with options to restore or permanently delete.

10. Frontend Guide - Components

client/src/components/Header/index.jsx

What This File Does

Header wrapper component that renders the Navbar. Used in the FrontendLayout.

```
import Navbar from './Navbar';

function Header() {
  return (
    <header className="header">
      <Navbar />
    </header>
  );
}

export default Header;
```

client/src/components/Header/Navbar.jsx

What This File Does

The main navigation bar. Shows logo, navigation links (Home, Shop, About, Contact), cart icon with item count badge, and user menu (login/register or profile/logout).

```
import { Link } from 'react-router-dom';
import { useAuth } from '../../context/Auth';
import { useCart } from '../../context/Cart';

function Navbar() {
  const { user, logout } = useAuth();
  const { itemCount } = useCart();

  return (
    <nav className="navbar">
      <Link to="/" className="logo">
        Bismillah General Store
      </Link>

      <ul className="nav-links">
        <li><Link to="/">Home</Link></li>
        <li><Link to="/shop">Shop</Link></li>
        <li><Link to="/about">About</Link></li>
        <li><Link to="/contact">Contact</Link></li>
      </ul>

      <div className="nav-actions">
        <Link to="/cart" className="cart-link">
          
          {itemCount > 0 && <span className="cart-badge">{itemCount}</span>}
        </Link>

        {user ? (
          <div className="user-menu">
            <span>Hello, {user.name}</span>
            {user.role === 'admin' && (
              <Link to="/dashboard">Dashboard</Link>
            )}
            <Link to="/orders">My Orders</Link>
            <button onClick={logout}>Logout</button>
          </div>
        ) : (
          <div className="auth-links">
            <Link to="/login">Login</Link>
            <Link to="/register" className="btn btn-
primary">Register</Link>
          </div>
        )}
      </div>
    </nav>
  );
}
```

```
    </nav>
  );
}
```

-> Auth.jsx (user state)

-> Cart.jsx (itemCount)

client/src/components/Footer/index.jsx

What This File Does

Footer wrapper component that renders the Copyright component. Used in the FrontendLayout.

client/src/components/Footer/Copyright.jsx

What This File Does

Displays copyright notice with store name and current year. May include links to privacy policy and terms of service.

```
function Copyright() {
  return (
    <footer className="copyright-footer">
      <p>&copy; {new Date().getFullYear()} Bismillah General Store. All
rights reserved.</p>
    </footer>
  );
}
```

client/src/components/Chat/ChatWidget.jsx

What This File Does

A floating chat widget that allows customers to send messages to admin support. Opens a chat window when clicked and allows sending/receiving messages.

```
import { useState, useEffect } from 'react';
import { useAuth } from '../../context/Auth';
import api from '../../config/api';

function ChatWidget() {
  const [isOpen, setIsOpen] = useState(false);
  const [messages, setMessages] = useState([]);
  const [newMessage, setNewMessage] = useState('');
  const { user } = useAuth();

  const sendMessage = async () => {
    if (!newMessage.trim()) return;
    await api.post('/chat/send', {
      message: newMessage,
      userId: user?._id,
    });
    setNewMessage('');
    fetchMessages();
  };

  return (
    <div className="chat-widget">
      <button className="chat-toggle" onClick={() => setIsOpen(!isOpen)}>

      </button>
      {isOpen && (
        <div className="chat-window">
          <h3>Chat with us</h3>
          <div className="messages">
            {messages.map((msg, i) => (
              <div key={i} className={msg.sender === 'user' ? 'user-
msg' : 'admin-msg'}>
                {msg.text}
              </div>
            ))}
          </div>
          <input
            value={newMessage}

```

```

        onChange={(e) => setNewMessage(e.target.value)}
        onKeyPress={(e) => e.key === 'Enter' && sendMessage()}
        placeholder="Type a message..."
      />
      <button onClick={sendMessage}>Send</button>
    </div>
  )}
</div>
);
}

```

client/src/components/Misc/ScreenLoader.jsx

What This File Does

Full-screen loading spinner shown while the application is initializing (verifying token, fetching categories).

```

function ScreenLoader() {
  return (
    <div className="screen-loader">
      <div className="spinner"></div>
      <p>Loading...</p>
    </div>
  );
}

```

client/src/components/Misc/NoPage.jsx

What This File Does

404 Not Found page displayed when a user navigates to an unknown route.

```
import { Link } from 'react-router-dom';

function NoPage() {
  return (
    <div className="no-page">
      <h1>404</h1>
      <p>Page not found</p>
      <Link to="/">Go back home</Link>
    </div>
  );
}
```

client/src/components/Misc/ProtectedRoute.jsx

What This File Does

Route guard component that protects routes requiring authentication. If isAdmin prop is set, also checks for admin role. Redirects to login if not authenticated.

```
import { Navigate, Outlet } from 'react-router-dom';
import { useAuth } from '../../context/Auth';

function ProtectedRoute({ isAdmin = false }) {
  const { user, loading } = useAuth();

  if (loading) return <div>Loading...</div>;

  // Not logged in - redirect to login
  if (!user) return <Navigate to="/login" replace />;

  // Admin route but user is not admin - redirect to home
  if (isAdmin && user.role !== 'admin') return <Navigate to="/" replace />;

  // Authorized - render child routes
  return <Outlet />;
}
```

How ProtectedRoute Works

This component does not render any UI itself. It uses `<Outlet />` to render child routes if authorized, or `<Navigate>` to redirect if not. The `replace` prop replaces the current history entry so the user cannot go back to the protected page.

11. Frontend Guide - Utilities

client/src/utils/format.js

What This File Does

Formatting utility functions for displaying prices, dates, and other data in a consistent format throughout the application.

```
// Format price in Pakistani Rupees
export const formatPrice = (price) => {
  return "Rs. " + Number(price).toLocaleString("en-PK", { minimumFractionDigits:
0 });
};

// Format date to readable string
export const formatDate = (dateString) => {
  const options = { year: "numeric", month: "short", day: "numeric" };
  return new Date(dateString).toLocaleDateString("en-US", options);
};

// Format date with time
export const formatDateTime = (dateString) => {
  const options = {
    year: "numeric", month: "short", day: "numeric",
    hour: "2-digit", minute: "2-digit"
  };
  return new Date(dateString).toLocaleDateString("en-US", options);
};

// Truncate text with ellipsis
export const truncateText = (text, maxLength = 50) => {
  if (text.length <= maxLength) return text;
  return text.substring(0, maxLength) + "...";
};
```

Usage: `formatPrice(250)` returns `"Rs. 250"`, `formatDate("2026-01-15")` returns `"Jan 15, 2026"`.

`client/src/utils/chat.js`

What This File Does

Utility functions for the chat feature including sending messages, fetching conversation history, and formatting chat timestamps.

```
import api from "../config/api";

export const fetchMessages = async (userId) => {
  const response = await api.get("/chat/messages/" + userId);
  return response.data;
};

export const sendMessage = async (userId, message) => {
  const response = await api.post("/chat/send", { userId, message });
  return response.data;
};

export const formatChatTime = (timestamp) => {
  return new Date(timestamp).toLocaleTimeString("en-US", {
    hour: "2-digit", minute: "2-digit"
  });
};
```

`client/src/utils/recycleBin.js`

What This File Does

Utility functions for the recycle bin feature including restoring soft-deleted items and permanently deleting them.

```
import api from "../config/api";
import toast from "react-hot-toast";
```

```
export const restoreItem = async (type, id) => {
  try {
    await api.put("/" + type + "/restore/" + id);
    toast.success("Item restored successfully");
    return true;
  } catch (error) {
    toast.error("Failed to restore item");
    return false;
  }
};

export const permanentDelete = async (type, id) => {
  try {
    await api.delete("/" + type + "/permanent/" + id);
    toast.success("Item permanently deleted");
    return true;
  } catch (error) {
    toast.error("Failed to delete item");
    return false;
  }
};

export const fetchDeletedItems = async (type) => {
  const response = await api.get("/" + type + "/deleted");
  return response.data;
};
```

12. Backend Guide - Server & Configuration

server/index.js

What This File Does

The main entry point of the Express server. Sets up middleware (CORS, JSON parsing), connects to MongoDB, mounts all route modules, and starts listening on a configured port.

```
const express = require("express");
const cors = require("cors");
const dotenv = require("dotenv");
const connectDB = require("../config/db");

dotenv.config();
connectDB();

const app = express();

app.use(cors({
  origin: process.env.CLIENT_URL || "http://localhost:5173",
  credentials: true,
}));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

app.use("/api/auth", require("../routes/auth"));
app.use("/api/products", require("../routes/products"));
app.use("/api/categories", require("../routes/categories"));
app.use("/api/orders", require("../routes/orders"));
app.use("/api/users", require("../routes/users"));
app.use("/api/dashboard", require("../routes/dashboard"));
app.use("/api/stock", require("../routes/stock"));
app.use("/api/chat", require("../routes/chat"));
app.use("/api/upload", require("../routes/upload"));
```

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ message: err.message || "Internal Server Error" });
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log("Server running on port " + PORT);
});
```

Line-by-Line Explanation

- **Lines 1-4:** Import Express, CORS, dotenv, and database connection
- **Lines 6-7:** Load .env file and connect to MongoDB
- **Lines 11-14:** CORS middleware - allows frontend to make requests to this server
- **Lines 15-16:** Body parsing middleware for JSON and form data
- **Lines 18-26:** Mount route modules at their respective paths
- **Lines 28-31:** Global error handler
- **Lines 33-36:** Start the server on configured port

-> config/db.js

-> routes/auth.js

-> routes/products.js

-> routes/categories.js

-> routes/orders.js

-> routes/users.js

-> routes/dashboard.js

-> routes/stock.js

-> routes/chat.js

-> routes/upload.js

Understanding Middleware Order

Express middleware runs in the order it is defined. CORS must come first to allow requests, then JSON parsing to read request bodies, then routes. The error handler must be last to catch any errors from routes.

`server/config/db.js`

What This File Does

Handles the MongoDB connection using Mongoose. Reads the connection URI from environment variables and establishes the connection.

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(
      process.env.MONGODB_URI || "mongodb://localhost:27017/bismillah-store"
    );
    console.log("MongoDB Connected: " + conn.connection.host);
  } catch (error) {
    console.error("Database connection error: " + error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

The MONGODB_URI should be set in your .env file. For local development, it defaults to mongodb://localhost:27017/bismillah-store.

`server/config/cloudinary.js`

What This File Does

Configures Cloudinary for image uploads. Cloudinary is a cloud service that handles image storage, optimization, and delivery via CDN.

```
const cloudinary = require("cloudinary").v2;
```

```
cloudinary.config({  
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,  
  api_key: process.env.CLOUDINARY_API_KEY,  
  api_secret: process.env.CLOUDINARY_API_SECRET,  
});  
  
module.exports = cloudinary;
```

Environment Variables Required

You need a Cloudinary account. Get your credentials from cloudinary.com dashboard and add them to your .env file.

13. Backend Guide - Middleware

server/middleware/auth.js

What This File Does

JWT authentication middleware. Verifies the JWT token from the Authorization header, decodes it, and attaches the user object to the request. Also provides role-based authorization functions.

```
const jwt = require("jsonwebtoken");
const User = require("../models/User");

const auth = async (req, res, next) => {
  try {
    const authHeader = req.header("Authorization");
    if (!authHeader || !authHeader.startsWith("Bearer ")) {
      return res.status(401).json({ message: "No token, authorization denied" });
    }

    const token = authHeader.replace("Bearer ", "");
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.id).select("-password");

    if (!user) {
      return res.status(401).json({ message: "Token is not valid" });
    }

    req.user = user;
    req.token = token;
    next();
  } catch (error) {
    res.status(401).json({ message: "Token is not valid" });
  }
};
```

```
const authorize = (...roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ message: "Access denied" });
    }
    next();
  };
};

module.exports = { auth, authorize };
```

How JWT Authentication Works

Step	What Happens	Where
1. Login	User sends email/password, server validates and creates JWT	POST /api/auth/login
2. Store Token	Frontend stores JWT in localStorage	client/src/context/Auth.jsx
3. Send Token	Axios interceptor adds token to Authorization header	client/src/config/api.js
4. Verify Token	auth middleware verifies JWT and attaches user to req	server/middleware/auth.js
5. Access Route	Route handler can use req.user for business logic	server/routes/*.js

-> models/User.js

<- All protected routes use this

server/middleware/upload.js

What This File Does

File upload middleware using Multer and Cloudinary. Handles multipart form data for image uploads and streams files directly to Cloudinary.

```
const multer = require("multer");
const cloudinary = require("../config/cloudinary");
const { CloudinaryStorage } = require("multer-storage-cloudinary");

const storage = new CloudinaryStorage({
  cloudinary: cloudinary,
  params: {
    folder: "bismillah-store",
    allowed_formats: ["jpg", "jpeg", "png", "webp"],
    transformation: [{ width: 800, height: 800, crop: "limit" }],
  },
});

const upload = multer({
  storage: storage,
  limits: { fileSize: 5 * 1024 * 1024 },
  fileFilter: (req, file, cb) => {
    if (file.mimetype.startsWith("image/")) {
      cb(null, true);
    } else {
      cb(new Error("Only image files are allowed"), false);
    }
  },
});

module.exports = upload;
```

Usage in routes: `router.post("/", auth, upload.single("image"), createProduct)` - The `upload.single("image")` middleware processes a single file upload from the "image" field.

14. Backend Guide - Models

Mongoose models define the schema (structure) of documents stored in MongoDB collections. Each model maps to a MongoDB collection.

server/models/User.js

What This File Does

Defines the User schema with fields for name, email, password (hashed), phone, role, and timestamps. Includes methods for password hashing and comparison.

```
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, "Name is required"],
    trim: true,
  },
  email: {
    type: String,
    required: [true, "Email is required"],
    unique: true,
    lowercase: true,
    trim: true,
  },
  password: {
    type: String,
    required: [true, "Password is required"],
    minlength: 6,
  },
  phone: {
    type: String,
    default: "",
  },
  role: {
    type: String,
    enum: ["customer", "admin"],
```

```

        default: "customer",
      },
      isDeleted: {
        type: Boolean,
        default: false,
      },
    }, { timestamps: true });

userSchema.pre("save", async function(next) {
  if (!this.isModified("password")) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

userSchema.methods.comparePassword = async function(enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model("User", userSchema);

```

Schema Fields Explained

Field	Type	Validation	Purpose
name	String	Required, trimmed	User's full name
email	String	Required, unique, lowercase	Login identifier
password	String	Required, min 6 chars	Hashed password (never stored plain)
phone	String	Optional	Contact number
role	String	Enum: customer/admin	Access level
isDeleted	Boolean	Default: false	Soft delete flag
timestamps	-	Auto	createdAt, updatedAt

Password Security

Passwords are NEVER stored in plain text. The pre("save") hook automatically hashes the password before saving. The comparePassword method is used during login to verify the entered password.

server/models/Product.js

What This File Does

Defines the Product schema with fields for name, description, price, category reference, image, stock quantity, featured status, and soft delete flag.

```
const mongoose = require("mongoose");

const productSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, "Product name is required"],
    trim: true,
  },
  description: { type: String, default: "" },
  price: { type: Number, required: true, min: 0 },
  category: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Category",
    required: true,
  },
  image: { type: String, default: "" },
  stock: { type: Number, default: 0, min: 0 },
  featured: { type: Boolean, default: false },
  isDeleted: { type: Boolean, default: false },
}, { timestamps: true });

productSchema.index({ name: "text", description: "text" });

module.exports = mongoose.model("Product", productSchema);
```

The ref: "Category" creates a relationship between Product and Category. When querying, you can use `.populate("category")` to get full category details.

server/models/Category.js

What This File Does

Defines the Category schema for product categories (e.g., Fruits, Vegetables, Dairy).

```
const mongoose = require("mongoose");

const categorySchema = new mongoose.Schema({
  name: { type: String, required: true, unique: true, trim: true },
  image: { type: String, default: "" },
  description: { type: String, default: "" },
  isDeleted: { type: Boolean, default: false },
}, { timestamps: true });

module.exports = mongoose.model("Category", categorySchema);
```

server/models/Order.js

What This File Does

Defines the Order schema with embedded items array, shipping address, payment method, order status, and total amount.

```
const mongoose = require("mongoose");

const orderItemSchema = new mongoose.Schema({
  product: { type: mongoose.Schema.Types.ObjectId, ref: "Product", required: true
```

```
    },
    name: String,
    price: Number,
    quantity: { type: Number, required: true, min: 1 },
    image: String,
  });

const orderSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  items: [orderItemSchema],
  shippingAddress: {
    fullName: String,
    address: String,
    city: String,
    phone: String,
  },
  paymentMethod: {
    type: String,
    required: true,
    enum: ["Cash on Delivery", "Credit Card", "Debit Card", "JazzCash",
"EasyPaisa"],
  },
  totalAmount: { type: Number, required: true },
  status: {
    type: String,
    enum: ["pending", "confirmed", "processing", "shipped", "delivered",
"cancelled"],
    default: "pending",
  },
  isPaid: { type: Boolean, default: false },
}, { timestamps: true });

module.exports = mongoose.model("Order", orderSchema);
```

Why Denormalize Order Items?

Notice that order items store the product name, price, and image directly. This is intentional - if a product is later edited or deleted, the order history should still show the original data at the time of purchase.

server/models/Chat.js

What This File Does

Defines the Chat schema for storing customer support conversations between users and admins.

```
const mongoose = require("mongoose");

const messageSchema = new mongoose.Schema({
  sender: { type: String, enum: ["user", "admin"], required: true },
  senderId: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
  text: { type: String, required: true },
  timestamp: { type: Date, default: Date.now },
});

const chatSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  messages: [messageSchema],
  isActive: { type: Boolean, default: true },
}, { timestamps: true });

module.exports = mongoose.model("Chat", chatSchema);
```

server/models/StockMovement.js

What This File Does

Defines the StockMovement schema for tracking inventory changes. Records every stock addition or removal with reason and quantity.

```
const mongoose = require("mongoose");

const stockMovementSchema = new mongoose.Schema({
  product: { type: mongoose.Schema.Types.ObjectId, ref: "Product", required: true },
  type: { type: String, enum: ["in", "out", "adjustment"], required: true },
  quantity: { type: Number, required: true },
  reason: { type: String, required: true },
});
```

```
reference: { type: String, default: "" },
performedBy: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
}, { timestamps: true });

module.exports = mongoose.model("StockMovement", stockMovementSchema);
```

Database Schema Relationships

```
User (1) ----> (Many) Order
Product (1) ----> (Many) OrderItem (embedded in Order)
Category (1) ----> (Many) Product
User (1) ----> (Many) Chat
Product (1) ----> (Many) StockMovement
```

15. Backend Guide - Routes

Express routes handle incoming HTTP requests and define API endpoints. Each route file maps to a specific resource.

server/routes/auth.js

What This File Does

Handles user authentication routes: register, login, and verify token.

```
const router = require("express").Router();
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const User = require("../models/User");
const { auth } = require("../middleware/auth");

// POST /api/auth/register
router.post("/register", async (req, res) => {
  try {
    const { name, email, password, phone } = req.body;
    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return res.status(400).json({ message: "Email already registered" });
    }
    const user = new User({ name, email, password, phone });
    await user.save();
    res.status(201).json({ message: "Registration successful" });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// POST /api/auth/login
router.post("/login", async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ email });
    if (!user) {
      return res.status(400).json({ message: "Invalid credentials" });
    }
  }
});
```

```

const isMatch = await user.comparePassword(password);
if (!isMatch) {
  return res.status(400).json({ message: "Invalid credentials" });
}
const token = jwt.sign(
  { id: user._id, role: user.role },
  process.env.JWT_SECRET,
  { expiresIn: "7d" }
);
res.json({
  token,
  user: { _id: user._id, name: user.name, email: user.email, role:
user.role },
});
} catch (error) {
  res.status(500).json({ message: error.message });
}
});

// GET /api/auth/verify
router.get("/verify", auth, async (req, res) => {
  res.json({ user: req.user });
});

module.exports = router;

```

API Endpoints Summary

Method	Path	Auth?	Description
POST	/api/auth/register	No	Create new user account
POST	/api/auth/login	No	Authenticate user, return JWT
GET	/api/auth/verify	Yes	Verify token, return user data

server/routes/products.js

What This File Does

CRUD routes for products: create, read (list with filters), update, soft delete, restore, and permanent delete.

```
const router = require("express").Router();
const Product = require("../models/Product");
const { auth, authorize } = require("../middleware/auth");
const upload = require("../middleware/upload");

// GET /api/products - Public (with search, filter, pagination)
router.get("/", async (req, res) => {
  try {
    const { search, category, sort, page = 1, limit = 10, featured } =
      req.query;

    let query = { isDeleted: false };
    if (search) query.name = new RegExp(search, "i");
    if (category) query.category = category;
    if (featured === "true") query.featured = true;

    const products = await Product.find(query)
      .populate("category", "name")
      .sort({ createdAt: -1 })
      .skip((page - 1) * limit)
      .limit(Number(limit));

    const total = await Product.countDocuments(query);
    res.json({ products, total, page: Number(page), pages: Math.ceil(total /
limit) });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// GET /api/products/:id - Public
router.get("/:id", async (req, res) => {
  try {
    const product = await Product.findById(req.params.id).populate("category");
    if (!product) return res.status(404).json({ message: "Product not found"
});
    res.json(product);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// POST /api/products - Admin only, with image upload
router.post("/", auth, authorize("admin"), upload.single("image"), async (req, res)
=> {
  try {
    const productData = req.body;
```

```
        if (req.file) productData.image = req.file.path;
        const product = new Product(productData);
        await product.save();
        res.status(201).json(product);
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
});

// PUT /api/products/:id - Admin only
router.put("/:id", auth, authorize("admin"), upload.single("image"), async (req, res) => {
    try {
        const updateData = req.body;
        if (req.file) updateData.image = req.file.path;
        const product = await Product.findByIdAndUpdate(req.params.id, updateData,
        { new: true });
        res.json(product);
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
});

// DELETE /api/products/:id - Soft delete
router.delete("/:id", auth, authorize("admin"), async (req, res) => {
    try {
        await Product.findByIdAndUpdate(req.params.id, { isDeleted: true });
        res.json({ message: "Product moved to recycle bin" });
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
});

module.exports = router;
```

server/routes/orders.js

What This File Does

Order management routes: create order, get user orders, get all orders (admin), update order status.

```
const router = require("express").Router();
const Order = require("../models/Order");
const { auth, authorize } = require("../middleware/auth");

// POST /api/orders - Create order
router.post("/", auth, async (req, res) => {
  try {
    const { items, shippingAddress, paymentMethod, totalAmount } = req.body;
    const order = new Order({
      user: req.user._id, items, shippingAddress, paymentMethod, totalAmount,
    });
    await order.save();
    res.status(201).json(order);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// GET /api/orders/mine - Get current user orders
router.get("/mine", auth, async (req, res) => {
  try {
    const orders = await Order.find({ user: req.user._id })
      .populate("items.product", "name image")
      .sort({ createdAt: -1 });
    res.json(orders);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// GET /api/orders - Get all orders (Admin)
router.get("/", auth, authorize("admin"), async (req, res) => {
  try {
    const orders = await Order.find()
      .populate("user", "name email")
      .populate("items.product", "name image")
      .sort({ createdAt: -1 });
    res.json(orders);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// PUT /api/orders/:id/status - Update status (Admin)
router.put("/:id/status", auth, authorize("admin"), async (req, res) => {
  try {
    const { status } = req.body;
    const order = await Order.findByIdAndUpdate(req.params.id, { status }, {
      new: true });
    res.json(order);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});
```

```
    }  
  });  
  
  module.exports = router;
```

server/routes/dashboard.js

What This File Does

Dashboard analytics route: aggregates data for total sales, orders, users, products, and chart data for Recharts.

```
const router = require("express").Router();  
const Order = require("../models/Order");  
const User = require("../models/User");  
const Product = require("../models/Product");  
const { auth, authorize } = require("../middleware/auth");  
  
// GET /api/dashboard/stats - Admin only  
router.get("/stats", auth, authorize("admin"), async (req, res) => {  
  try {  
    const totalOrders = await Order.countDocuments();  
    const totalUsers = await User.countDocuments({ role: "customer" });  
    const totalProducts = await Product.countDocuments({ isDeleted: false });  
  
    const salesResult = await Order.aggregate([  
      { $match: { status: { $ne: "cancelled" } } },  
      { $group: { _id: null, total: { $sum: "$totalAmount" } } },  
    ]);  
    const totalSales = salesResult[0]?.total || 0;  
  
    const recentOrders = await Order.find()  
      .populate("user", "name")  
      .sort({ createdAt: -1 }).limit(5);  
  
    const salesData = await Order.aggregate([  
      { $group: {  
        _id: { $dateToString: { format: "%Y-%m", date: "$createdAt" } },  
        sales: { $sum: "$totalAmount" },  
      }},  
      { $sort: { _id: 1 } },  
      { $limit: 12 },  
    ]);  
  }  
});
```

```
    });

    const orderStatusData = await Order.aggregate([
      { $group: { _id: "$status", count: { $sum: 1 } } },
    ]);

    res.json({ totalSales, totalOrders, totalUsers, totalProducts,
recentOrders, salesData, orderStatusData });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

module.exports = router;
```

server/routes/categories.js

What This File Does

CRUD routes for categories: list all, create, update, and delete.

server/routes/users.js

What This File Does

User management routes for admins: list users, get user details, update user status.

server/routes/stock.js

What This File Does

Stock management routes: record stock movements, view stock history, adjust inventory levels.

`server/routes/chat.js`

What This File Does

Chat routes: send messages, fetch conversation history, get all active chats (admin).

`server/routes/upload.js`

What This File Does

Standalone file upload route for uploading images without associating them with a specific resource.

Complete API Routes Summary

Method	Path	Auth	Role	Description
POST	/api/auth/register	No	-	Register new user
POST	/api/auth/login	No	-	Login user
GET	/api/auth/verify	Yes	Any	Verify JWT token
GET	/api/products	No	-	List products
GET	/api/products/:id	No	-	Get product detail
POST	/api/products	Yes	Admin	Create product

Method	Path	Auth	Role	Description
PUT	/api/products/:id	Yes	Admin	Update product
DELETE	/api/products/:id	Yes	Admin	Soft delete product
GET	/api/categories	No	-	List categories
POST	/api/categories	Yes	Admin	Create category
POST	/api/orders	Yes	Customer	Create order
GET	/api/orders/mine	Yes	Customer	Get my orders
GET	/api/orders	Yes	Admin	Get all orders
PUT	/api/orders/:id/status	Yes	Admin	Update order status
GET	/api/dashboard/stats	Yes	Admin	Get analytics
GET	/api/users	Yes	Admin	List users
GET	/api/stock	Yes	Admin	Stock movements
POST	/api/chat/send	Yes	Any	Send message
GET	/api/chat/messages/:userId	Yes	Any	Get messages
POST	/api/upload	Yes	Admin	Upload file

16. How to Run the Project

Prerequisites

- **Node.js** (v16 or higher) - Download from nodejs.org
- **MongoDB** - Local installation or MongoDB Atlas cloud account
- **Cloudinary Account** - For image uploads (free tier available)
- **Git** - For cloning the repository

Step 1: Clone the Repository

```
git clone https://github.com/your-username/bismillah-general-store.git
cd bismillah-general-store
```

Step 2: Set Up Environment Variables

Create a `.env` file in the `server/` directory:

```
PORT=5000
MONGODB_URI=mongodb://localhost:27017/bismillah-store
JWT_SECRET=your-secret-key-here-make-it-strong
CLIENT_URL=http://localhost:5173
CLOUDINARY_CLOUD_NAME=your-cloud-name
CLOUDINARY_API_KEY=your-api-key
CLOUDINARY_API_SECRET=your-api-secret
```

Security Warning

Never commit your `.env` file to Git. Add it to `.gitignore`. The `JWT_SECRET` should be a long, random string.

Step 3: Install Backend Dependencies

```
cd server  
npm install
```

Step 4: Install Frontend Dependencies

```
cd ../client  
npm install
```

Step 5: Start MongoDB

If using local MongoDB, run `mongod` in terminal. Or use MongoDB Atlas and update MONGODB_URI in .env.

Step 6: Start the Backend Server

```
cd server  
npm run dev
```

Server should start on <http://localhost:5000>

Step 7: Start the Frontend Dev Server

```
cd client  
npm run dev
```

Frontend should start on <http://localhost:5173>

Step 8: Open in Browser

Navigate to <http://localhost:5173>

Creating an Admin User

Register a new account through the UI, then manually update the user role in MongoDB shell to "admin" using the `db.users.updateOne` command with the `$set` operator.

17. Common Concepts for Beginners

1. What is React?

React is a JavaScript library for building user interfaces. It uses **components** (reusable UI pieces) and **state** (data that changes over time) to create dynamic web pages.

Key React Concepts

Components: Functions that return JSX (HTML-like syntax). **State:** Data stored in components using `useState`. **Props:** Data passed from parent to child. **Effects:** Side effects using `useEffect` (API calls, subscriptions).

2. What is Node.js and Express?

Node.js lets you run JavaScript on the server. **Express** is a web framework that makes it easy to create APIs with routing and middleware.

3. What is MongoDB?

A **NoSQL database** that stores data in flexible JSON-like documents. Unlike traditional SQL databases, it does not require a fixed schema.

4. What is JWT?

JSON Web Token is a secure way to transmit information between client and server. After login, the server creates a JWT that the client sends with every request to prove identity.

JWT Flow:

1. User logs in with email/password
2. Server validates credentials
3. Server creates JWT with user ID and role
4. Client stores JWT in localStorage
5. Client sends JWT in Authorization header
6. Server verifies JWT on protected routes

5. What is REST API?

A **Representational State Transfer** API uses HTTP methods to perform operations:

Method	Purpose	Example
GET	Read data	GET /api/products
POST	Create data	POST /api/products
PUT	Update data	PUT /api/products/123
DELETE	Delete data	DELETE /api/products/123

6. What is Middleware?

Functions that run before route handlers. Used for authentication, logging, error handling, etc. Express processes middleware in order.

7. What is CORS?

Cross-Origin Resource Sharing - a security feature that blocks requests from different origins by default. The CORS middleware allows the frontend to make requests to the backend.

8. What is Mongoose?

An ODM (Object Document Modeling) library for MongoDB. It provides schema-based models, validation, and easy query building.

9. What is Soft Delete?

Instead of permanently deleting records, we set an `isDeleted: true` flag. This allows recovery from a recycle bin and preserves data integrity for related records.

10. What is SCSS?

A CSS preprocessor that adds features like variables, nesting, mixins, and functions. It compiles to regular CSS that browsers can read.

18. Tips for New Developers

Tip 1: Start Small

Do not try to understand everything at once. Start with the entry files (main.jsx, App.jsx) and follow the component tree down. Read one file at a time.

Tip 2: Use console.log

Add console.log statements to understand data flow. Check the browser console and terminal for output.

Tip 3: Read Error Messages

Error messages tell you exactly what went wrong. Read them carefully - they usually include the file name and line number.

Tip 4: Use Browser DevTools

Open Chrome DevTools (F12) to inspect elements, check Network requests, view console logs, and debug JavaScript.

Tip 5: Understand the Data Flow

Follow this pattern: User Action -> Component State Change -> API Call -> Server Processes -> Database Query -> Response -> UI Updates.

Common Mistakes to Avoid

- Forgetting to add `async/await` before API calls
- Not handling errors in `try/catch` blocks
- Hardcoding URLs instead of using environment variables
- Committing `.env` files with secrets to Git
- Not checking if a user is authenticated before accessing protected routes
- Modifying state directly instead of using `setState` functions

Recommended Learning Path

1. Learn JavaScript basics (variables, functions, arrays, objects, `async/await`)
2. Learn React fundamentals (components, state, props, effects)
3. Learn Express.js basics (routing, middleware, request/response)
4. Learn MongoDB/Mongoose (schemas, CRUD operations)
5. Study this project file by file

Debugging Checklist

When something is not working:

1. Check browser console for JavaScript errors
2. Check terminal/server console for backend errors
3. Verify API endpoints using Postman or browser

4. Check Network tab in DevTools for failed requests
5. Verify environment variables are set correctly
6. Check MongoDB connection status